

Shifting Automated Code Optimization Paradigms with Large Language Models

Anonymous Authors

Anonymous Institution

Anonymous City, Anonymous Country

Email: anonymous@example.com

Abstract—Performance code optimization is a vital aspect of modern software development, as it enables faster response times and reduced resource usage. These optimizations require a deep understanding of low-level hardware details and the intricacies of parallel processing, making them challenging even for experienced developers. With the advent of Large Language Models (LLMs), which are increasingly capable of generating and understanding code, there is growing interest in incorporating these models into automated code optimization processes. Traditionally, this automation involves transcribing the source code into a domain-specific representation that can be auto-tuned using grid search or machine learning algorithms without explicit human intervention, at the cost of following strict rules that ensure automated verifiability. LLMs also incorporate high-level semantics of the code and can thus perform transformations that go beyond verifiable automated optimizations. This paper investigates whether the traditional abstractions used in automated code optimization improve the performance and correctness of LLM-guided loop optimizations. We evaluate this using the PolyBench benchmark suite and demonstrate that LLMs, when provided with specific optimization goals, achieve better performance and correctness while generating C code compared to creating computation pipelines and schedules for established tuning frameworks, suggesting a potential shift in the approach to code optimization in the era of LLMs.

Index Terms—large language model, code optimization, high-performance computing, tuning frameworks, prompt strategies.

I. INTRODUCTION

Performance optimization of code is an essential part of software development, ensuring quick response times and efficient utilization of computational resources. This challenge is particularly pronounced in high-performance computing (HPC) applications that often involve complex computations over large datasets. This challenge is addressed on multiple levels, from low-level optimizations performed by compilers like GCC [1] and LLVM [2], and tools based on polyhedral models [3], [4], to high-level algorithmic optimizations typically performed by human experts or auto-tuning frameworks, such as OpenTuner [5]. However, achieving optimal performance requires a deep understanding of the underlying hardware (e.g., memory hierarchies and parallelism ranging from SIMD to distributed computing) and the specific characteristics of the target application, such as exploitable invariants or optimizable memory access patterns [6], [7].

Common optimization techniques that target memory hierarchies and parallelism include loop transformations such

as tiling, loop interchange or skewing, unrolling, fusion, and parallelization strategies (like OpenMP directives or SIMD vectorization). They are typically applied by the aforementioned polyhedral model-based frameworks or human experts, and then fine-tuned using auto-tuning frameworks. However, without deep expertise in hardware architectures and the mathematical models underlying these optimizations, applying these methods often leads to suboptimal performance [8].

To address this, domain-specific languages (DSLs) and frameworks, such as Halide [9], Exo [10], and Noarr [11], have been developed to simplify the process of specifying and tuning high-performance code for human programmers, replacing mathematical affine models with higher-level abstractions that better capture the semantics of computations and the intent of the optimizations. These frameworks also provide mechanisms for automatic tuning of the selected optimizations and their hyperparameters [12], [13], which enables writing performance-critical code to target specific hardware architectures without having to explicitly manage low-level details and re-implement the program for each target architecture separately.

In practice, applying these methods still requires human expertise to prepare optimization strategies that can be, through auto-tuning, adapted to the specific hardware and application characteristics — such as applying tiling with the tile sizes later determined by the auto-tuner, or specifying which loops can be safely parallelized. Alternative approaches based on polyhedral model optimizations are limited by the strict mathematical rules they must follow to ensure verifiability of the transformations and by speculative cost models. Additionally, the auto-tuning processes that explore the space of optimizations require significant computational resources since they need to measure the performance (and verify the correctness) of each candidate optimization.

Large language models (LLMs) have recently shown great promise in understanding and generating code, with models such as Codex [14], AlphaCode [15], and Code Llama [16] demonstrating state-of-the-art results. Most of these works focus on web development tasks or general programming tasks, often in languages such as Python or JavaScript that are not typically used for performance-critical applications. However, LLMs have also been shown to be capable of generating high-performance code [17], [18] and even optimizing existing code, usually by proposing optimization directives that are

easily applicable and verifiable by a formal toolchain (e.g., adding OpenMP pragmas for parallelization) [19], [20]. These approaches, however, have limited expressiveness and may not achieve the best possible performance on tasks that require more complex or non-trivial optimizations, often struggling with correctness in parallel contexts [21]. Recent works have explored using LLMs to either generate or optimize high-performance code using specialized prompting [22]–[25], but these works typically focus on specific languages or frameworks.

Our work aims to explore the capabilities of LLMs across multiple domain-specific languages for high-performance code tuning and optimization, as well as across different prompting strategies for generating the optimized code. The goal is to determine whether LLMs can effectively leverage the abstractions provided by these frameworks, or whether generating optimized C code directly leads to better performance and correctness of the produced code.

We formulate the goals of this study in the following research questions (RQs):

- RQ1 Do state-of-the-art frameworks for loop optimizations offer any benefits (e.g., correctness verification) for LLM-guided optimization over direct code generation?
- RQ2 Do more detailed prompts improve the performance and correctness of LLM-guided loop optimizations?
- RQ3 Does creating an abstract optimization plan improve the performance and correctness of LLM-guided loop optimizations?
- RQ4 How do the approaches discussed in the previous questions compare in terms of performance gain, result correctness, and overall prompt costs?
- RQ5 Can LLMs apply code optimizations that are not easily expressible in existing DSLs and frameworks reliably without changing the semantics of the optimized algorithms?

To study these research questions in a meaningful way, we explicitly choose not to run the requests in an iterative process where the LLM can refine its outputs based on feedback from the compiler or runtime results. While such an approach is proven to be effective in practice [26]–[28], it also introduces additional variables that can obscure the effects of the studied factors. However, when applying the findings of this study in practice, such iterative refinement can certainly be combined with the best-performing approaches identified here to further enhance the optimization results.

Answering the presented question should help to outline guidelines for effectively using LLMs for high-performance code optimization and provide insights into the strengths and limitations of current DSLs and frameworks in this context. In addition, we experimented with associated tasks for LLMs, most notably generating code baselines for selected DSLs and frameworks from an original C implementation of the algorithms. This should help extend our work to other tasks in the future.

The paper is organized as follows. Section II provides a brief overview of the common optimization techniques in the context of the studied DSLs and frameworks. Our proposed approach to employing LLMs for code optimizations is formulated in Section III. The evaluation setup and experimental results are summarized in Section IV. Section V discusses related work, and Section VI concludes the paper.

Additionally, we provide a replication package [29] with the entire codebase, prompts, and experiment results¹.

II. BACKGROUND

Loop transformations and data layout transformations are common techniques, but they can be challenging to apply correctly and effectively. Let us demonstrate this on a well-known example — the naïve matrix multiplication. Mathematically, it can be defined as $C' = C + A \times B$ (or, element-wise, $c'_{i,j} = c_{i,j} + \sum_k a_{i,k} b_{k,j}$). Where C , A , and B are matrices of sizes $M \times N$, $M \times K$, and $K \times N$, respectively. An implementation in C may look like the following:

```
1 void matmul(int M, int N, int K,
2            float *C, float *A, float *B)
3 {
4     for (int i = 0; i < M; i++) {
5         for (int j = 0; j < N; j++) {
6             for (int k = 0; k < K; k++) {
7                 C[i*N + j] += A[i*K + k] * B[k*N + j];
8             }
9         }
10    }
```

Listing 1: Naïve matrix multiplication in C

Applying loop tiling with tile sizes of 32 for both the i and j loops, changing the B matrix layout to be column-major for better memory access patterns, and unrolling the innermost k loop by a factor of 4 will have the following result:

```
1 for (int io = 0; io < M; io += 32) { // Tiled i loop
2     for (int jo = 0; jo < N; jo += 32) { // Tiled j loop
3         for (int i = io; i < min(io+32, M); i++) { // Inner i
4             for (int j = jo; j < min(jo+32, N); j++) { // Inner j
5                 for (int k = 0; k < K; k += 4) { // Unrolled k loop
6                     C[i*N + j] += A[i*K + k] * B[j*K + k];
7                     C[i*N + j] += A[i*K + (k + 1)] * B[j*K + (k+1)];
8                     C[i*N + j] += A[i*K + (k + 2)] * B[j*K + (k+2)];
9                     C[i*N + j] += A[i*K + (k + 3)] * B[j*K + (k+3)];
10                }
11            }
12        }
13    }
```

Listing 2: Tiled and unrolled matrix multiplication in C

Such optimization, which can be further followed by parallelization using OpenMP or SIMD vectorization, leads to a dramatic increase in code complexity, making it significantly more error-prone and harder to maintain. For this reason, various frameworks and domain-specific languages (DSLs) have been developed to help express the algorithm and its transformations more easily and safely, without having to manually rewrite the whole code for each optimization.

Examples of such frameworks that we selected as good representatives of different approaches are Halide [9], Exo [10], and Noarr [11]. Although this list is not exhaustive, these frameworks cover a sufficient range for our LLM experiments to address RQ1.

¹The link will be changed to a non-anonymous upon acceptance.

In the following sections, we give a brief overview of the three frameworks. All three frameworks aim to facilitate the development of high-performance code through various abstractions and optimizations, and allow the user to (automatically) tune and parallelize the computations. We illustrate the basic usage of each framework using the previously introduced example of naïve matrix multiplication.

A. Halide

Halide [9] is a domain-specific language (DSL) embedded in C++ primarily designed for image processing and stencil computations. An example code snippet for matrix multiplication using Halide is shown in Listing 3.

```

1 Buffer<float> C(M, N), A(M, K), B(K, N);
2 Func matmul;
3 Var i, j;
4 RDom k(0, K);
5
6 // Algorithm definition
7 matmul(i, j) = C(i, j);
8 matmul(i, j) += A(i, k) * B(k, j);
9
10 // Schedule definition
11 Var i_inner, j_inner;
12 matmul.update().tile(i, j, i_inner, j_inner, 32, 32)
13   .unroll(k, 4)
14   .vectorize(k);
15
16 matmul.realize(C);

```

Listing 3: Halide example code for matrix multiplication with scheduling

The core concept in Halide is the separation of the algorithm from its execution schedule. The algorithm is defined as a computation pipeline broken down into a series of *functions* (Func in Halide) that describe how to compute the output values based on the input values. Each function is defined over a multi-dimensional domain using *variables* (Var in Halide) and can involve *reduction domains* (RDom in Halide) for operations like summations.

Following such a definition of a computation pipeline, Halide allows specifying various optimizations through a user-defined *schedule* that describes how the computation should be executed. In the example above, we explicitly apply loop tiling on the second (update) stage of the `matmul` function by splitting the iteration space of `i` and `j` into 32×32 , unroll the loop over the `k` dimension and request the SIMD vectorization `usi`. The schedule is defined separately from the algorithm itself and must follow certain rules that ensure verifiability of the transformations. Even with these restrictions, the scheduling language is expressive enough to achieve significant performance improvements for a wide range of applications [9], [12].

Since Halide is specifically designed for stencil-like computations, it would be expected that the framework performs well on such tasks. Since TVM [30], based on Halide, has been adopted in industry for deep learning workloads, Halide is also expected to perform well on matrix multiplication tasks. However, its expressivity is quite limited, and implementing more complex algorithms in it is challenging, often requiring non-trivial workarounds that break the computation into many

smaller Halide functions or stages of functions. While such workarounds can help express the algorithm in Halide, they also increase the amount of performance overhead that has to be mitigated by Halide’s optimizations and the user-defined schedules.

B. Exo

Exo [10] is a Python-based framework for generating high-performance code through program synthesis and optimization. An example code snippet for matrix multiplication using Exo is shown in Listing 4.

```

1 @proc
2 def kernel_gemm(
3     ni: size,
4     nj: size,
5     nk: size,
6     C: float[ni, nj] @ DRAM,
7     A: float[ni, nk] @ DRAM,
8     B: float[nk, nj] @ DRAM,
9 ):
10     for i in seq(0, ni):
11         for j in seq(0, nj):
12             for k in seq(0, nk):
13                 C[i, j] += A[i, k] * B[k, j]

```

Listing 4: Exo example code for matrix multiplication

The core concept in Exo is defining computations as *procedures* (`proc` in Exo) that describe the operations to be performed on the input data. The procedures can include standard control flow constructs such as loops (`for` in Exo) and conditionals, allowing for a more general representation of algorithms compared to Halide; however, the language still strictly separates data computation from bounds and memory layout specifications — for example, conditionals based on data values are not allowed. This ensures that the Exo engine can reason about the computation and apply optimizations safely, while sacrificing some flexibility in algorithm expression.

Similar to Halide, Exo allows specifying various optimizations through externally specified *rewrite rules* [10]. These rules are applied using pattern matching to specific operations or loop nests within the procedure, together with performing type inference, allowing the Exo engine to verify the correctness of the transformations before generating optimized C code.

Contrasting with Halide, Exo is designed for general loop-based computations. After performing the specified optimization rewrites and verifications, Exo uses the resulting structure to generate the optimized C code — if there are no optimizations applied, the expected generated C code for the above specification would result from simply replacing the specific Python syntax with C syntax.

The loop-based structure of Exo makes it more flexible than Halide, allowing it to perfectly express imperfectly nested loops, which is Halide’s bane. However, with flexibility comes complexity. Thus, the optimizations available in Exo are more limited compared to Halide, unless the user provides additional information about the computation semantics to help Exo perform more complex transformations (such as providing

assertions about loop bounds to enable transformations such as tiling).

C. Noarr

Noarr [11] is a C++ header-only library that provides abstractions for defining data layouts and algorithms. An example code snippet for matrix multiplication using Noarr is shown in Listing 5.

```

1 using V = float;
2 auto C = bag(scalar<V>() ^ vector<'i'>(M) ^ vector<'j'>(N));
3 auto A = bag(scalar<V>() ^ vector<'i'>(M) ^ vector<'k'>(K));
4 auto B = bag(scalar<V>() ^ vector<'k'>(K) ^ vector<'j'>(N));
5
6 traverser(C, A, B) | [] (auto idx) {
7     C[idx] += A[idx] * B[idx];
8 };

```

Listing 5: Noarr example code for matrix multiplication

Among the selected frameworks, Noarr uses the most lightweight and straightforward approach. It is a C++ library that provides template-based abstractions for defining data structures and performing computations on them; thus, it does not impose any specific syntax or semantic restrictions on the user code beyond standard C++. The core concept in Noarr is defining data structures using composable abstractions (like `vector` or `scalar`) that describe the layout and dimensions of the data, and performing computations using *traversers* that iterate over the iteration space defined by the closure over their dimensions. Noarr does not perform any semantic analysis of the computations, but imposes safety via type checking and dimension naming.

Since Noarr involves no external code generation or runtime support on top of C++, it is expected to behave similarly to hand-written C/C++ code for the same algorithm, given that the algorithm semantics are identical. Compared to Halide and Exo, Noarr is simpler in terms of setup and integration into existing C++ projects, but it lacks capabilities such as applying more complex transformations or performing verification. It also differs in that it abstracts indexation (e.g., indexation such as `C[i, j]` in Exo or Halide is represented as `C[idx]` in Noarr with no explicit indices). This, combined with dimensions of the structures and loop nests being named rather than positional, helps to avoid certain classes of bugs related to incorrect indexing. It is thus expected that Noarr implementations will be less error-prone compared to analogs in Halide or Exo. However, since Noarr does not provide verification capabilities, the errors in Noarr will likely be caught only at runtime (e.g., incorrect results) rather than at compile time.

D. PolyBench

To test and evaluate our LLM-guided methods, we need to select and implement a set of coding problems that could benefit from HPC optimizations. The PolyBench [31] benchmark suite is a collection of 30 numerical computations commonly used in scientific computing and high-performance computing research. It includes a variety of algorithms from different

domains, such as linear algebra, data mining, stencil computations, and others. All benchmarks are implemented in plain C, and the suite defines several problem sizes (MINI, SMALL, MEDIUM, LARGE, and EXTRALARGE) — in total, there are 150 benchmark variants for a given data type (e.g., `float` or `double`). The PolyBench suite is widely used for evaluating the performance of compilers, optimization techniques, and hardware architectures [3], [4], [17], [26], [32], [33], so we consider it a suitable choice for our experiments.

III. METHODOLOGY

In our experiments, we instruct the LLM to automatically optimize the performance of the given problem while preserving the original semantics. Doing so in pure C code is challenging even for human programmers. As illustrated in Listing 2, even a relatively simple optimization, such as performing tiling across the input dimensions of a matrix multiplication, changing a layout of the involved arrays, and applying unrolling and vectorization, increases the code complexity considerably.

The selected PolyBench provides only C baseline implementations of the coding problems. To test it with different frameworks and DSLs, we need to translate the code, preferably as a part of the automated process. We have used an LLM to transcribe the code, and we provide this step as a part of our methodology, allowing our approach to be easily extended to other DSLs and frameworks in the future. Furthermore, using LLMs should help to mitigate possible human bias or mistakes, namely the tendency to apply optimizations to the baseline code, which could taint the subsequent automated optimization process or its evaluation.

The baseline code is subjected to two types of LLM-guided optimizations: *direct* code optimizations and optimizations based on an *abstract optimization plan*. In the direct optimizations, we try multiple prompts with different levels of detail. The abstract optimization plan approach is divided into two subsequent prompts — the first asks LLMs to generate an abstract optimization plan, and the second applies generated plans to produce the optimized code. The idea is to help the LLM to better structure its thoughts before applying them to the code [34]. Furthermore, the abstract plan may be easier to interpret and analyze for humans, allowing for a better understanding of the optimizations applied by the LLM.

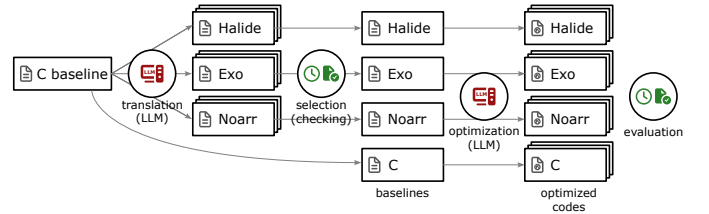


Figure 1: Methodology overview

A complete diagram of our methodology is shown in Figure 1. We address the individual parts in the following sections, while the technical details (including the prompts) are provided in the replication package [29].

A. Translation

The LLM is requested to translate the baseline C code (a benchmark code from the PolyBench suite) into the target framework (or DSL) code while preserving the behavior of the original as closely as possible. Ideally, the generated code follows the same structure as the original and performs the same computations in the same order (i.e., having the same memory access patterns), but using the constructs provided by the target DSL/framework.

In each request, the LLM is given (in the system message) the programming guide of the target framework, which is short enough to fit into the LLM’s context window, and specific enough to give the LLM a good idea of how to use the framework. Since the guides do not include complete reference documentation, we also provide the LLM with an example of translating the GEMM benchmark from PolyBench into the target framework, and a list of framework-specific rules that the LLM should follow when generating the code. The goal of these additional resources is to further explain how to use the target framework correctly and to help the LLM avoid common pitfalls. We tailor the rules to each framework individually, based on our prior experience, and evaluate their effectiveness and sufficiency on a small subset of problems before proceeding to the full evaluation. We have used a representative set of four problems to tune the translation prompts: GEMM, 2MM, Floyd-Warshall, and Heat-3D.

We make five independent translation requests for each benchmark and framework combination to account for the variability in the LLM’s outputs. From the successfully translated outputs that pass a verification step (described in Section III-D), we then select the one that best matches the original code in performance over the SMALL, MEDIUM, and LARGE input sizes from PolyBench. The algorithm that passes the verification and matches the original behavior most closely is then used in subsequent optimization experiments.

B. Direct optimization

The direct optimization approach is similar to the translation step. The LLM is given the baseline code (in pure C or translated by the previous step), along with the programming guide and specific rules of the respective framework (if applicable). We make five independent optimization requests for each benchmark and framework combination (including the C code).

The running system is briefly described in general terms as a modern x86-64 machine with a given number of CPU cores, along with the compiler version, the optimization flags, and the target framework standard specification. The LLM is then requested to optimize the code for performance while preserving its semantics. The prompts are tuned on the same representative set of four benchmarks as the translation prompts, as described in Section III-A. The optimization goals are specified by asking the LLM to improve data locality, computation efficiency, and memory access patterns, while preserving the original semantics of the code and the boiler-

plate code used for setting up the benchmarks and measuring their execution time.

We use three different prompting strategies in our endeavor to answer RQ2:

- **Naïve approach:** The LLM is simply asked to optimize the provided code for performance while preserving its semantics without additional guidance.
- **All-hints approach:** The LLM is provided with specific optimization hints (detailed below) to guide the optimization process.
- **Choose-hints approach:** Is similar to the all-hints approach, but the LLM is given an additional instruction to choose the most suitable hints from the provided ones and ignore the rest.

The optimization hints provided to the LLM are divided into four categories. These categories were originally intended to be provided separately to the LLM to conduct more fine-grained experiments with the prompt level of detail, but our preliminary experiments showed that it is better to leave the selection of the relevant hints to the LLM itself.

- **Cache:** Directs the LLM to apply tiling to the loops and reorganize the data layouts of the involved arrays to improve cache locality.
- **Structure:** Directs the LLM to define temporary buffers of limited size (up to 50% of the original arrays) to store intermediate results.
- **Arithmetic:** Directs the LLM to consider applying algebraic transformations to reduce the number of arithmetic operations performed in the code. Also suggests using accumulator variables to minimize memory accesses within loops.
- **Parallelism:** Directs the LLM to identify parallelization opportunities in the code and use appropriate constructs from the target DSL (or framework) to parallelize the computations across multiple CPU cores.

Each of these hints further asserts that the LLM should ensure that the optimizations preserve the original semantics of the code.

C. Abstract optimization plan

To address RQ3, we also investigate an alternative approach where the LLM is first asked to describe an abstract optimization plan without modifying the code, and then, in a second request, to choose from multiple optimization plans and apply the selected one to the original code. This approach should force the LLM into deeper reasoning about the optimizations before applying them to the code, potentially leading to better-structured and more effective optimizations [26].

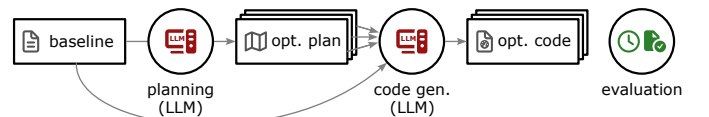


Figure 2: Abstract plan optimization overview

Figure 2 illustrates the abstract optimization plan approach. The first request is based on the direct `all-hints` approach, where the LLM is provided with the same four optimization hints and is asked to describe an optimization plan based on these hints. It differs in that the request does not contain the framework-specific rules, and the LLM is instructed to describe the optimizations in abstract terms for later application. Similarly to the translation step, the prompt also includes an example of an abstract optimization plan for the GEMM benchmark that shows the expected format and level of detail of the optimization plan.

The optimization plan is based on similar principles as the abstractions in Halide and Exo — namely, separating the data structures, computation pipelines, and optimization schedules. The LLM is expected to describe (in separate sections) the domain of the computation (dimensions), the involved data structures and their layouts, any further algorithm parameters, and the computational steps to be performed (via loop nests representing quantification, and conditional blocks), and any relevant data dependencies between the computational steps. Then, the LLM is expected to describe the optimizations to be applied to the computation in terms of transformations of the described dimensions, data structures, and computational steps via descriptive directives similar to those in the studied DSLs — such as tiling, loop interchange, unrolling, vectorization, and parallelization. Unlike in verifiability-focused approaches, the LLM is allowed to define further directives as needed to express the optimizations not covered by the provided ones.

The system message of the second request is based on the naïve direct optimization prompts with additional instructions to select the best plan from the previous request and apply it to the baseline code instead of giving general optimization instructions. Similarly to the first request, the expected output is showcased on the GEMM benchmark in the prompt (using the target framework). The LLM is then provided with a user message containing the baseline code and the previously generated optimization plans from the first request.

D. Verification

We verify the correctness of the generated code by comparing its output against the output of the original unoptimized C code over the PolyBench-provided input sizes. We consider the generated code to be correct if its output matches the original code’s output within a small numerical tolerance for all tested input sizes. If the generated code fails to compile or run for any input size, or produces incorrect output for any input size, it is considered incorrect. We also impose a time limit for the execution set to 8 minutes, which safely exceeds the expected execution time of any benchmark on our hardware.

While this verification approach does not guarantee full semantic equivalence between the original and generated code, there is not a significant risk of false positives as the benchmarks are intentionally designed with non-symmetric input data and dimension sizes non-divisible by common tiling factors, making it unlikely that an incorrect solution would pass over all tested input sizes.

E. Threats to validity

One of the most common threats to validity when dealing with LLM outputs is their inherent stochasticity. We mitigate this by making multiple independent requests and by considering the similarity of results from multiple different approaches in the analysis.

A related threat is the LLM’s sensitivity to prompt wording and structure, which we mitigate by aligning the structure of the prompts with the provider’s prompting best practices [35] and by tuning the prompts on a representative set of benchmarks (Section III-A) before proceeding to the full evaluation. We put great effort into minimizing any unintentional bias in the prompts by constructing them in such a way that they do not favor any particular approach or framework, and they offer comparable levels of guidance and information across all approaches. Furthermore, we used the Grammarly tool to standardize the language quality and style across all prompts.

Finally, the LLM’s familiarity with the benchmarks and their optimizations and with the studied DSLs may also threaten the validity. We mitigate this by including multiple DSLs that differ in their design and usage, and appending their respective programming guides, along with examples and LLM-specific rules, to the prompts. Based on the experiments used to tune the prompts (Section III-A), we observed that the LLM’s competence is both sufficient and consistent across all studied DSLs, with no significant correlation found between performance and the framework’s popularity or age.

IV. EVALUATION

The evaluation was performed on a dual-socket system equipped with Intel Xeon Gold 6130 processors, each comprising 16 cores with 2-way hyper-threading (64 threads in total). All benchmarks were compiled with GCC 15.2 and CMake 4.12, enabling OpenMP and using the `-O3` optimization flag along with `-march=native` and `-mtune=native` for architecture-specific compiler optimizations. The performance is measured in seconds using a wall clock timer, as specified in the PolyBench benchmark suite [31].

For the LLM requests, we used GPT-5.1 (thinking) [36] with the reasoning effort set to `high` (which represented the state of the art for our use case at the time of the experiments). The requests were processed through the official OpenAI Batch API, with five independent requests sent in each batch. This setup allows for parallel processing and ensures at least a 50% reduction in cost.

For most performance results, we report the `speedup@5` metric, defined as the maximum speedup achieved across five independent code generation attempts. To calculate the time for a single attempt, we averaged the execution time over three runs with two warm-up runs (which ensures insignificant variance). We selected this *best-of-five* approach to reflect a practical workflow where a developer generates multiple candidate solutions and selects the most efficient one, mitigating the inherent non-determinism of LLM outputs [14], while keeping the number of attempts manageable.

Since code generation does not guarantee correctness, we verified each generated implementation by comparing its output against the C baseline. Any implementation that failed to produce matching results on any input size was deemed incorrect and excluded from the performance measurements. Furthermore, any implementation that failed to finish within 8 minutes was treated as incorrect — this limit is generous enough to accommodate even the most suboptimal valid implementation.

A. Translation results

Figure 3 compares the performance of the PolyBench benchmarks translated by LLM into Exo, Halide, and Noarr frameworks against the original C code without any optimizations applied. The x -axis represents the different benchmarks with horizontal lines separating each benchmark category as defined by the PolyBench suite. The y -axis shows the logarithmic speedup achieved by each translation, with baseline denoted by the horizontal line at $y = 0$. The y dimension of the displayed points shows an aggregated speedup over all input sizes defined by PolyBench for each benchmark, using the geometric mean following standard practice [26].

The plot shows that most translations fall within a reasonable range of the original C performance. Most translations achieving worse performance than the C baseline belong to the Halide framework. This trend extends to most benchmarks in the stencil category, even though Halide is designed specifically for stencil-like computations — the PolyBench stencil benchmarks are iterative and they are not easily expressible in Halide’s functional paradigm. However, the assumption that Halide should perform well on matrix multiplication tasks holds true: for the 2MM, 3MM, `symm`, and `trmm` benchmarks, the LLM occasionally generates Halide implementations that outperform the C baseline by a significant margin.

The LLM consistently fails to successfully translate the `cholesky` and `nussinov` benchmarks to Noarr and Halide. For these benchmarks, the LLM must correctly rewrite the output formatting; however, it incorrectly falls back to a generic pattern that dumps the entire output array (which is invalid for these specific algorithms). For the `gramschmidt` benchmark specifically, all LLM implementations fail due to numerical instability and arithmetic errors for all three frameworks.

As the translation to the Exo framework is typically the most straightforward, the LLM achieves the highest success rate with Exo, producing only 12 incorrect translations out of 150 attempts (this number includes compile-time errors, runtime errors, and correctness failures). For Noarr, the number of incorrect translations is 30. For Halide, the number of incorrect translations is 67, representing nearly half of all translation attempts. However, a higher failure rate is expected for Halide, as translating certain computations requires expressing them as a chain of function updates, or, in some cases, numerous intermediate functions, which can easily lead to code that may be technically correct, but is extremely slow (and times out across all input sizes). Overall, the results roughly correspond

to the expressiveness and complexity of transcribing the source computations to each target framework.

B. Optimization Results

Figure 4 illustrates the performance improvements achieved by different optimization approaches, displayed as a heatmap of speedups over the unoptimized C baseline. Each row represents one framework/approach, and each column represents one benchmark. The color intensity indicates the magnitude of the speedup, with darker colors representing higher speedups. Blank cells indicate that no correct optimization was produced for that benchmark using the given approach. In each column (benchmark), the best-performing approach is highlighted with a red border. The `speedup@5` metric is used, representing the best speedup achieved among 5 independent LLM optimization attempts.

We have chosen `EXTRALARGE` as it models a case where a performance typically matters the most. In our replication package [29], we provide the same plots for all test sizes (from `MINI` to `EXTRALARGE`) and also plot aggregating all sizes using a geometric mean. The relative speedups usually follow the same trend across the sizes.

The results indicate that the `all-hints` approach applied to direct C code generation achieves the best overall performance across most benchmarks, and the `choose-hints` approach performs quite similarly. This similarity persists across all frameworks, indicating that providing specific optimization hints to the LLM leads to better performance and that the approach is robust regarding the specific prompt wording. Most notably, there is no indication that the LLM would pursue unsuitable optimization paths more frequently for the `all-hints` approach compared to the less assertive `choose-hints` approach; however, the former still tends to yield slightly more aggressive optimizations.

Contrary to initial expectations, the `from-plan` approach does not appear to provide significant performance benefits over the other two approaches, or even over the naïve approach. Surprisingly, it offers good performance specifically for the `solvers` category of benchmarks, when optimizing the C code directly; however, the difference is not significant enough to distinguish it from stochastic variation.

For the framework-based approaches, there is lower variability between the different optimization strategies. The `all-hints` and `choose-hints` variants still perform similarly, with `all-hints` tending to perform best; however, for the Exo and Noarr frameworks, the `naïve` approach performs comparably well. This suggests that for these frameworks, the performance differences are driven more by the LLM stochasticity than by the specific optimization approach. For the Halide framework, the results are too sparse to draw meaningful conclusions.

Figure 5 shows the accompanying correctness results for the optimization approaches displayed in Figure 4. It reveals that although specific optimization requests lead to more aggressive transformations, which typically increases the risk of errors, the correctness rates for the `all-hints` and

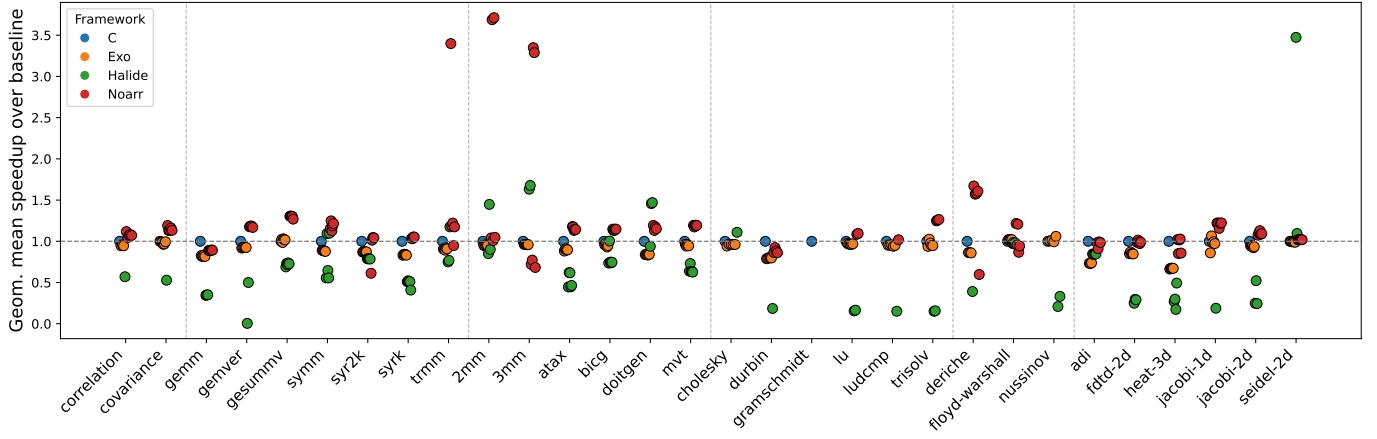


Figure 3: Performance of PolyBench benchmarks translated to Exo, Halide, and Noarr frameworks measured as speedup over the original C code, aggregated over all input sizes using the geometric mean. Each point represents one LLM translation of a given benchmark.

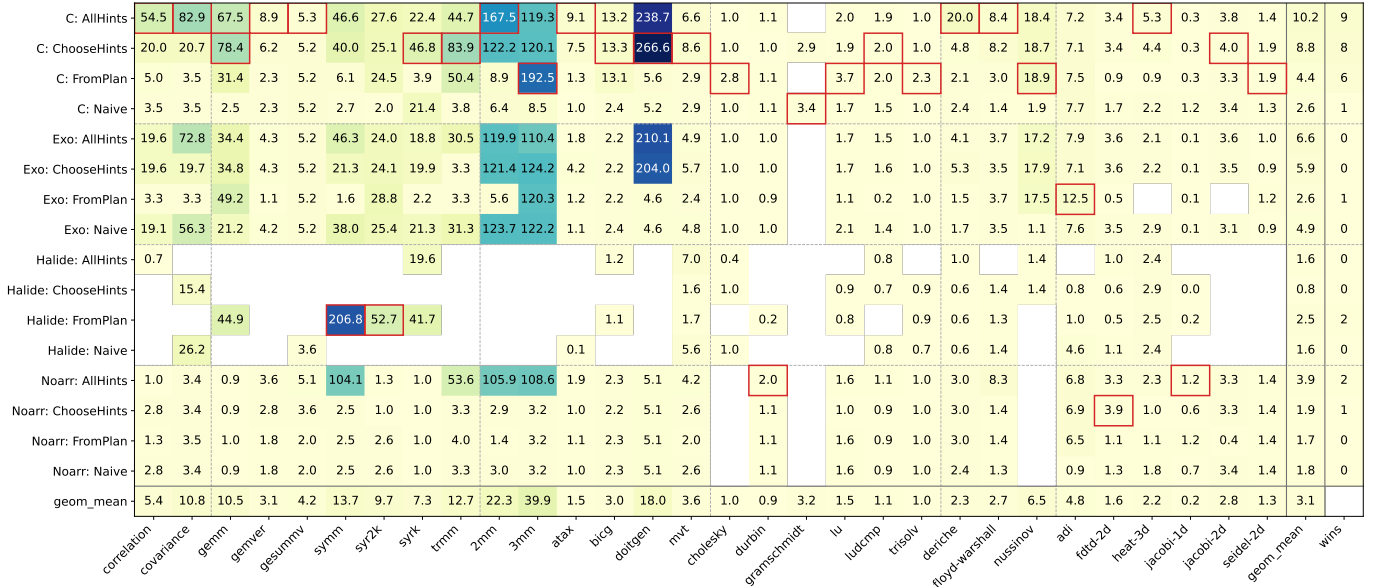


Figure 4: Performance improvements achieved by different optimization approaches for the EXTRALARGE input sizes of the benchmarks, displayed as a heatmap of speedups over the C baseline. The speedup@5 metric is used.

choose-hints approaches remain generally comparable to the naïve approach. Surprisingly, the from-plan approach tends to exhibit a slightly lower correctness rate than all other approaches for the frameworks, contradicting the hypothesis that an abstract optimization plan would help the LLM avoid invalid optimization paths.

C. Comparison to prior work

We compare our best-performing approach (all-hints applied to direct C code generation) to prior work on LLM-guided optimizations, specifically the approach presented by Merouani et al. [26]. Their work, which is the most recent study addressing similar goals, employs an iterative agentic framework to refine optimizations based on feedback from

compiler and runtime measurements. They demonstrated impressive performance improvements of 200× or above on 2MM, 3MM, correlation, covariance, syr2k, syr, and trmm, with less significant improvements on other PolyBench benchmarks at EXTRALARGE input sizes. However, this was achieved at the substantial cost of running up to 150 LLM requests and 150 compilation/execution cycles per benchmark².

Our results show that even when using a non-iterative approach with only 5 LLM requests per benchmark and selecting the best optimization among them, it is possible to achieve

²It should be noted that they compare their results against a Tiramisu re-implementation of the benchmarks. We have compared our results with tiramisu as well, plot is similar to Figure 4 and can be found in the replication package [29].

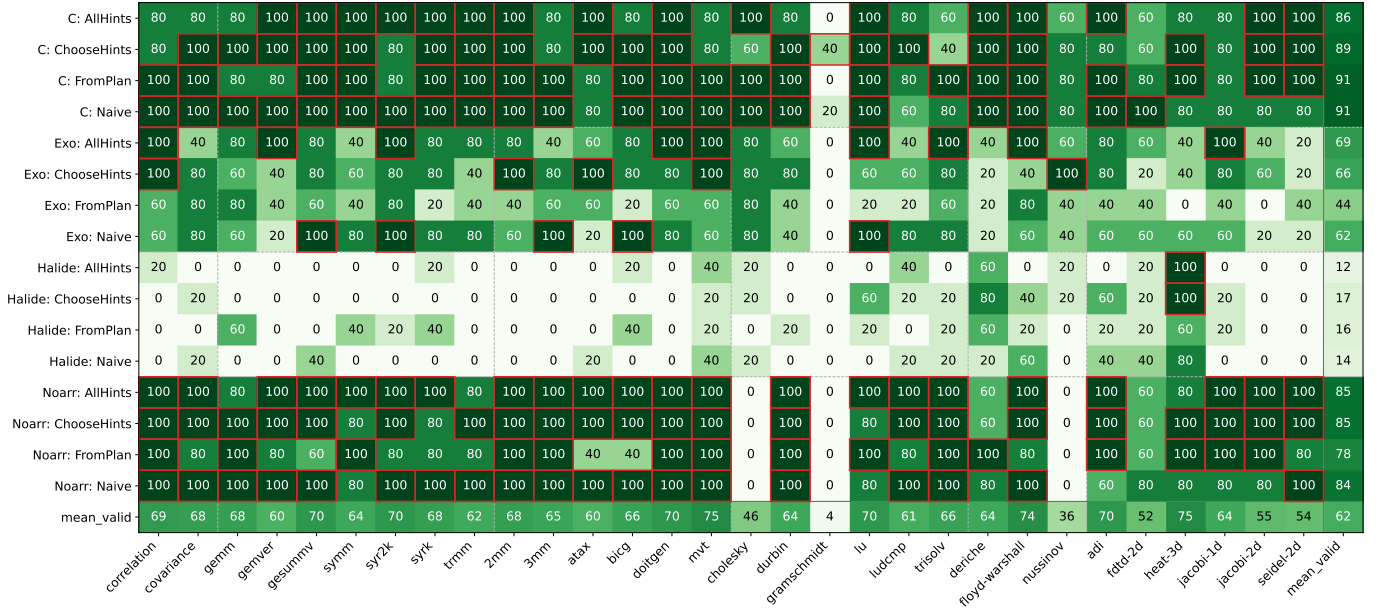


Figure 5: Correctness achieved by different optimization approaches displayed as a heatmap of valid optimizations over total attempts. Percentage values indicate the correctness rate (percentage of correct optimizations among 5 attempts) for each benchmark and approach.

results of the same order of magnitude on most of these benchmarks (namely `correlation`, `covariance`, `2MM`, and `3MM`). Furthermore, we observe significant speedups on benchmarks where their approach performed poorly, such as `doitgen`, `symm`, `fdtd-2d`, `heat-3d`, and `jacobi-2d`. Compared to Merouani et al., the average number of output tokens over the optimization process drops from 1’000’000 for 5 attempts with 30 iterations [26] to just 91’965 tokens for the 5 attempts without iterating, making the process 10.9× more cost-effective in terms of LLM usage.

While their work demonstrates the potential of iterative refinement for LLM-guided optimizations, our findings indicate that a careful choice of the abstraction framework the LLM is requested to use, combined with an effective prompting strategy, yields significant performance improvements even without iterative feedback. This suggests that the choice of abstraction and prompt design plays a crucial role in the success of LLM-guided code optimizations.

We also observe much higher correctness rates in our results compared to their work. While this could be attributed to the difference in the strength of the LLM models used, we also note a success rate on Exo and Halide translations that is quite comparable to the correctness rates observed by Merouani et al. [26], whose approach uses an abstraction that generalizes Halide. This suggests that the incidence of incorrect optimizations may also be attributed to the characteristics of the abstractions used.

D. Answering research questions

RQ1: Do frameworks help? Section IV-A clearly shows that, for most benchmarks, using the frameworks in the context

of LLM-guided optimizations does not provide significant performance or correctness benefits over direct C code generation. It should be noted, however, that identifying an incorrect optimization in a specifically designed framework can be easier than in C code, as shown by Merouani et al. [26]. Conversely, these frameworks often impose strict rules that can significantly limit the optimization opportunities the LLM can explore compared to direct C code generation.

RQ2: Do specific hints help? Section IV-B indicates that providing specific optimization goals to the LLM generally leads to better performance of the optimized code compared to naïve optimization requests. This trend is consistent across both framework-based and direct C code generation approaches.

RQ3: Does the abstract plan help? The goal of this research question was to determine if a two-step optimization process (first, generating an abstract optimization plan, then implementing it) improves optimization quality. We hypothesized that allowing the LLM to first propose multiple abstract strategies and subsequently select the most promising one to implement would reduce the likelihood of pursuing invalid or suboptimal paths. Unlike the approach in RQ2, which relied on externally provided hints during code generation, this method relies on the LLM creating benchmark-specific optimization strategies prior to writing code.

Unexpectedly, our results show that the introduction of this abstract planning phase does not improve performance in most cases. It is our belief that the current LLM generation has grown beyond the need of being explicitly forced into deep reasoning (which is the main point of the `from-plan` approach), and it can perform such reasoning implicitly (within

a single prompt) when provided with detailed hints. Further investigation needed to confirm this hypothesis is out of the scope of this work, but we observe that the LLM measurably adapts its reasoning depth based on the given benchmark.

RQ4: Comparison of all approaches. From the results obtained for RQ1–RQ3, it is evident that the specificity of optimization goals (RQ2) has the most significant impact on the performance of LLM-guided loop optimizations. However, it should be noted that the further development of abstraction frameworks tailored for LLM-guided optimizations could potentially alter this conclusion in the future.

RQ5: Do LLMs surpass verifiability? The results discussed in Section IV-C indicate that LLMs can indeed produce optimizations that are not easily expressible in existing frameworks focused on verifiability. This should not be interpreted as verifiability being unattainable for such optimizations, but rather that current frameworks impose strict rules limiting the optimization opportunities and, by extension, the performance achievable with LLM-guided optimizations. Future work should explore developing methods to achieve verifiability through alternative means.

V. RELATED WORK

Automated code optimization has historically relied on analytical frameworks, most notably the polyhedral model. Tools such as Pluto [4] and Polly [3] leverage mathematical abstractions to perform aggressive loop reorganizations, including tiling, skewing, and interchange. While effective, these techniques are constrained by strict static analysis requirements that limit their ability to apply speculative or semantically complex optimizations.

For greater flexibility, domain-specific languages (DSLs) like Halide [9], Exo [10], Noarr [11], and Tiramisu [37] decouple algorithms from execution schedules. This separation facilitates auto-tuning, where schedulers search for optimal configurations using beam search or learned cost models [12], [13]. However, these approaches are often computationally demanding, especially when targeting diverse hardware backends.

The advent of Large Language Models (LLMs) has transformed code generation [16], [38]. Despite their success in general programming, empirical studies indicate that off-the-shelf models struggle to produce efficient parallel code for high-performance computing (HPC) tasks [18], [21], [25]. To bridge this gap, researchers have explored various enhancement strategies.

One direction is fine-tuning specialized models. Examples include HPCCoder [39], which targets parallel code generation, and the LM4HPC framework [19], [20] for OpenMP pragmas. Beyond standard fine-tuning, reinforcement learning (RL) has been applied to discover novel algorithms (AlphaDev [40]) or to align models with performance feedback (RLPF [17]). However, these methods are resource-intensive, require access to proprietary weights, and risk obsolescence as foundation models evolve.

Alternatively, researchers utilize models via prompt engineering. Studies show that expert-crafted hints, source-to-source directives, and zero-shot reasoning strategies can significantly improve performance without retraining [22]–[24], [34]. The limitation here is the reliance on user expertise to formulate effective prompts.

To automate this process, agentic frameworks employ iterative feedback loops using compiler or runtime signals to refine outputs [27], [28]. A prominent example is the work by Merouani et al. [26], which integrates LLMs with Tiramisu to generate verifiable schedules. While robust, such agentic approaches could be very costly, often requiring dozens of compilation cycles (and prompts) per task.

We leverage generalized optimization hints and generating optimization plans to guide the LLM, achieving competitive speedups via cost-efficient, non-iterative prompting. Compared to prior works, our approach focuses on investigating multiple DSLs and frameworks, as well as direct C code generation, and different prompting strategies to identify the most effective methods for LLM-guided code optimization.

VI. CONCLUSION

In this work, we present a comprehensive study on using large language models (LLMs) to optimize code for high-performance applications in conjunction with frameworks that aim to ease the optimization process for both human programmers and automated tuning systems. Our experiments demonstrate that LLMs are more capable of generating efficient code without utilizing these frameworks, both in terms of performance and code correctness. We also showed that, for contemporary LLMs, the specificity of the prompt plays a more significant role than whether the LLM is tasked to optimize code directly in C/C++ or through tools for user-guided optimization such as Exo, Halide, or Noarr.

This has two important implications. First, it suggests that the current generation of LLMs has reached a level of proficiency that surpasses the use of verifiable DSLs and frameworks for code optimization. Second, it indicates that future research should focus on specifically tailoring DSLs that will be better aligned with the capabilities of LLMs, potentially leading to new paradigms in code optimization that leverage the strengths of both approaches.

ACKNOWLEDGMENT

This paper was supported by the Johannes Amos Comenius Programme (P JAC, Natural and anthropogenic georisks) project CZ.02.01.01/00/22_008/0004605 and by Charles University institutional funding SVV (grant 260821).

REFERENCES

- [1] R. M. Stallman *et al.*, “Using the GNU Compiler Collection,” *Free Software Foundation*, vol. 4, no. 02, 2003.
- [2] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 2004, pp. 75–86.
- [3] T. Grosser, A. Groesslinger, and C. Lengauer, “Polly—performing polyhedral optimizations on a low-level intermediate representation,” *Parallel Processing Letters*, vol. 22, no. 04, p. 1250010, 2012.

- [4] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, "Pluto: A practical and fully automatic polyhedral program optimization system," in *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation (PLDI 08)*, Tucson, AZ (June 2008). CiteSeer, vol. 146, 2008.
- [5] J. Ansel, S. Kamil, K. Veeramachaneni, J. Ragan-Kelley, J. Bosboom, U.-M. O'Reilly, and S. Amarasinghe, "OpenTuner: An extensible framework for program autotuning," in *Proceedings of the 23rd international conference on Parallel architectures and compilation*, 2014, pp. 303–316.
- [6] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [7] K. Kennedy and J. R. Allen, *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc., 2001.
- [8] M. F. Abdulla, "Manual and fast C code optimization," *arXiv preprint arXiv:1203.0681*, 2012.
- [9] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *Acm Sigplan Notices*, vol. 48, no. 6, pp. 519–530, 2013.
- [10] Y. Ikarashi, G. L. Bernstein, A. Reinking, H. Genc, and J. Ragan-Kelley, "Exocompilation for productive programming of hardware accelerators," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022, pp. 703–718.
- [11] J. Klepl, A. Šmelko, L. Rozsypal, and M. Kruliš, "Abstractions for C++ code optimizations in parallel high-performance applications," *Parallel Computing*, p. 103096, 2024.
- [12] R. T. Mullaipudi, A. Adams, D. Sharlet, J. Ragan-Kelley, and K. Fatahalian, "Automatically scheduling halide image processing pipelines," *ACM Transactions on Graphics (TOG)*, vol. 35, no. 4, pp. 1–11, 2016.
- [13] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T.-M. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand *et al.*, "Learning to optimize Halide with tree search and random programs," *ACM Transactions on Graphics (TOG)*, vol. 38, no. 4, pp. 1–12, 2019.
- [14] M. Chen, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [15] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [16] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [17] D. Nichols, P. Polasam, H. Menon, A. Marathe, T. Gamblin, and A. Bhatele, "Performance-aligned LLMs for generating fast code," *arXiv preprint arXiv:2404.18864*, 2024.
- [18] P. Valero-Lara, A. Huante, M. Al Lail, W. F. Godoy, K. Teranishi, P. Balaprakash, and J. S. Vetter, "Comparing Llama-2 and GPT-3 LLMs for HPC kernels generation," in *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 2023, pp. 20–32.
- [19] L. Chen, A. Bhattacharjee, N. Ahmed, N. Hasabnis, G. Oren, V. Vo, and A. Jannesari, "OMPGPT: A generative pre-trained transformer model for OpenMP," in *European Conference on Parallel Processing*. Springer, 2024, pp. 121–134.
- [20] L. Chen, P.-H. Lin, T. Vanderbruggen, C. Liao, M. Emani, and B. De Supinski, "LM4HPC: Towards effective language model application in high-performance computing," in *International Workshop on OpenMP*. Springer, 2023, pp. 18–33.
- [21] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?" in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, 2024, pp. 281–294.
- [22] M. Brabec, J. Klepl, M. Töpfer, and M. Kruliš, "Tutoring LLM into a better CUDA optimizer," in *European Conference on Parallel Processing*. Springer, 2025, pp. 250–263.
- [23] M. Palkowski and M. Gruzewski, "GPT-driven source-to-source transformation for generating compilable parallel CUDA code for Nussinov's algorithm," *Electronics*, p. 488, 2024.
- [24] B. Chen, N. Mustakin, A. Hoang, S. Fuad, and D. Wong, "VSCuda: LLM based CUDA extension for Visual Studio Code," in *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 11–17.
- [25] W. F. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. S. Vetter, "Large language model evaluation for high-performance computing software development," *Concurrency and Computation: Practice and Experience*, p. e8269, 2024.
- [26] M. Merouani, I. K. Bernou, and R. Baghdadi, "Agentic auto-scheduling: An experimental study of LLM-guided loop optimization," *arXiv preprint arXiv:2511.00592*, 2025.
- [27] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.
- [28] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," *arXiv preprint arXiv:2304.05128*, 2023.
- [29] A. Anonymous, "Replication package repository," 2025. [Online]. Available: https://anonymous.4open.science/r/2026-isc-llm_code_optimization-D3A5
- [30] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "TVM: An automated End-to-End optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594.
- [31] L.-N. Pouchet and T. Yuki, "PolyBench/C 4.2.1 (beta)," 2016. [Online]. Available: <https://sourceforge.net/projects/polybench/files/polybench-c-4.2.1-beta.tar.gz/download>
- [32] S. Verdoolaege, J. Carlos Juega, A. Cohen, J. Ignacio Gomez, C. Tenllado, and F. Cathoor, "Polyhedral parallel code generation for CUDA," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 9, no. 4, pp. 1–23, 2013.
- [33] A. H. Ashouri, G. Mariani, G. Palermo, E. Park, J. Cavazos, and C. Silvano, "COBAYN: Compiler autotuning framework using bayesian networks," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 13, no. 2, pp. 1–25, 2016.
- [34] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, pp. 22 199–22 213, 2022.
- [35] OpenAI, "Prompt engineering guide," <https://platform.openai.com/docs/guides/prompt-engineering>, 2024, accessed: 2025-12-19.
- [36] —, "GPT-5.1: A smarter, more conversational chatgpt," 2025. [Online]. Available: <https://openai.com/research/gpt-5-1>
- [37] R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe, "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2019, pp. 193–205.
- [38] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *arXiv preprint arXiv:2406.00515*, 2024.
- [39] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, "HPC-Coder: Modeling parallel programs using large language models," in *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. Prometheus GmbH, 2024, pp. 1–12.
- [40] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern *et al.*, "Faster sorting algorithms discovered using deep reinforcement learning," *Nature*, vol. 618, no. 7964, pp. 257–263, 2023.